

Talking to the Phone

Everything about `adb`, from what it actually is to the commands that found the bug in NoShorts. Written against the exact machine and handset this project runs on, so every command here has been executed for real.

Written for	Chandradithya Janaswami
Assumes	You can open a terminal. Nothing else.
Toolchain	platform-tools 37.0.1 at F:\platform-tools
Handset	OnePlus CPH2717 · Android 16 · ColorOS

CONTENTS

01	What adb actually is	07	Settings, permissions, appops
02	Installing it, and the PATH	08	logcat, and when it goes silent
03	Pairing: USB debugging and the RSA key	09	Reading the screen: uiautomator
04	Devices, states, and when it lies	10	Wireless adb
05	Moving apps and files	11	Every command this project used
06	adb shell — a terminal on the phone	12	Troubleshooting, and staying safe

PART 01

What adb actually is

`adb` is the Android Debug Bridge. Most people treat it as "the command that installs APKs", which undersells it: it is a general-purpose channel into a running Android device, and once you understand its shape the odd failure modes stop being mysterious.

Three pieces, not one

Typing `adb` starts a chain of three separate programs.

The client	<code>adb.exe</code> , the thing you actually type. It does almost nothing itself: it packages your command and sends it to the server.
The server	A background process on your PC, listening on TCP port 5037 . It keeps track of every connected device and routes commands. You never start it by hand; the first <code>adb</code> command starts it for you.
The daemon	<code>adb</code> , running on the phone. It receives commands and executes them with the privileges of the <code>shell</code> user.

That is why the very first command you run prints something like this, and later ones do not:

The server starting itself

```
$ adb devices
* daemon not running; starting now at tcp:5037
* daemon started successfully
List of devices attached
```

CONCEPT

Because the server is a separate long-lived process, it can get into a bad state that restarting your terminal will not fix. `adb kill-server` followed by `adb start-server` is the correct "turn it off and on again", and it resolves a surprising share of problems.

What it can and cannot do

`adb` runs as the `shell` user (uid 2000) on a retail phone. That account is far more powerful than an app but well short of root.

CAN	CANNOT
Install and remove apps	Read another app's private data
Change secure system settings	Modify system partitions
Grant runtime permissions	Become root on a retail build
Read logs, dump system state	Bypass the lock screen
Inject taps and keystrokes	Do anything while the phone is off

`adb root` exists but only works on `userdebug` or `eng` builds. On your CPH2717, a retail `user` build, it will refuse. That is normal and not something to fight.

Installing it, and the PATH

adb ships in a bundle called **platform-tools**. You do not need Android Studio, the SDK manager, or a Google account. It is a 7 MB zip with no installer.

Windows, PowerShell – what we actually ran

```
Invoke-WebRequest `
  -Uri 'https://dl.google.com/android/repository/platform-tools-latest-windows.zip' `
  -OutFile "$env:TEMP\platform-tools.zip"

Expand-Archive -Path "$env:TEMP\platform-tools.zip" -DestinationPath 'F:\' -Force

F:\platform-tools\adb.exe version
```

Google publishes the same bundle for every platform, always at a "latest" URL, so there is no version to pick:

OS	URL SUFFIX
Windows	platform-tools-latest-windows.zip
macOS	platform-tools-latest-darwin.zip
Linux	platform-tools-latest-linux.zip

Putting adb on the PATH

Right now you must type the full path every time. **PATH** is a list of folders your shell searches when you type a bare command; adding the folder makes `adb` work from anywhere.

Permanent, for your user only – no admin needed

```
$old = [Environment]::GetEnvironmentVariable('Path', 'User')
[Environment]::SetEnvironmentVariable('Path', "$old;F:\platform-tools", 'User')
```

Open a *new* terminal afterwards. A running shell reads PATH once at startup and will not see the change.

GOTCHA – MIXING SHELLS ON WINDOWS

You have both PowerShell and Git Bash here, and they disagree about paths. Git Bash rewrites anything that looks like a Unix path before handing it to a Windows program, so `adb shell uiautomator dump /sdcard/ui.xml` becomes `C:/Program Files/Git/sdcard/ui.xml` and fails. This bit us during the NoShorts debugging. Two fixes: prefix the command with `MSYS_NO_PATHCONV=1`, or double the leading slash as `//sdcard/ui.xml`. In PowerShell the problem does not exist.

Pairing: USB debugging and the RSA key

A phone will not talk to adb until you explicitly allow it. Three steps, once per device.

1. Unlock Developer options

Settings, About phone, then tap **Build number** seven times. A counter appears and then confirms you are a developer. This has been the ritual since 2012 and every manufacturer keeps it, though they move the screen around. On ColorOS it is under Settings, About device, Version.

2. Turn on USB debugging

Settings, System, Developer options, **USB debugging**.

OPPO, OnePlus, Realme and Xiaomi devices also have a separate **Install via USB** toggle. Without it, `adb install` fails while everything else works, which is a genuinely confusing failure. Some Xiaomi builds require a signed-in account before they let you flip it.

3. Accept the RSA fingerprint

Plug in the phone. A dialog shows a fingerprint and asks whether to allow debugging. Tick **Always allow from this computer** and accept.

Under the hood: adb generated a keypair the first time it ran, at `~/.android/adbkey` and `adbkey.pub`. Accepting stores your public key on the phone in `/data/misc/adb/adb_keys`. From then on, authentication is automatic.

GOTCHA — THE DIALOG NEVER APPEARS

Almost always a cable problem. Many USB cables are charge-only: four wires for power, no data pair. The phone charges, the PC sees nothing, and there is no error to read. Try a different cable before anything else. If the cable is good, pull down the USB notification and switch the mode from "Charging" to **File transfer**. If the dialog was dismissed once, revoke and replug: Developer options, **Revoke USB debugging authorisations**.

Devices, states, and when it lies

```
The command you run before every other command

$ adb devices -l
List of devices attached
ZDEE4LHQ5LMJ8DEI   device product:CPH2717IN model:CPH2717 device:0P613BL1 transport_id:1
```

The first column is the device serial. The second is the **state**, and it is the single most useful diagnostic adb gives you.

STATE	MEANING	FIX
device	Ready.	Nothing.
unauthorized	Connected, but you never accepted the RSA prompt.	Look at the phone screen and accept.
offline	Seen but not responding. Often a half-woken device or a stale server.	adb kill-server , replug.
no permissions	Linux only: your user cannot open the USB device.	Add a udev rule.
(nothing listed)	Cable, USB mode, or debugging is off.	Part 03.

More than one device

With two devices attached, every command errors until you say which one. Use `-s` with the serial:

```
adb -s ZDEE4LHQ5LMJ8DEI shell wm size
adb -d shell ...      # the only USB device
adb -e shell ...      # the only emulator
```

Moving apps and files

Installing

install

```
adb install app.apk           # fresh install
adb install -r app.apk       # reinstall, keep the app's data
adb install -g app.apk       # grant all runtime permissions up front
adb install -d app.apk       # allow a version downgrade
adb install -r -d --no-streaming app.apk  # for stubborn OEM builds
```

Installing over adb bypasses the Play Protect consent dialog entirely. Play Protect still scans the app and may complain afterwards, but it cannot block the install. That is why adb was the answer when sideloading NoShorts through the file manager was refused.

GOTCHA — INSTALL_FAILED_UPDATE_INCOMPATIBLE

Android refuses to update an app when the new APK is signed with a different key than the installed one. This is a security guarantee, not a bug, and there is no flag to override it. You must uninstall first, which wipes the app's data. We hit this on every NoShorts release because each CI run generates a fresh debug keystore. The real fix is one stable signing key reused across builds.

uninstall

```
adb uninstall com.noshorts.blocker
adb uninstall -k com.noshorts.blocker  # remove app, keep data and cache
```

Files

pull and push — device path first for pull, last for push

```
adb pull /sdcard/ui.xml C:\temp\ui.xml
adb push C:\temp\config.json /sdcard/Download/

adb shell ls /sdcard/
adb shell rm /sdcard/ui.xml
```

`/sdcard` is shared storage, readable and writable by the shell user. App-private storage under `/data/data/<package>` is not, unless the app is debuggable, in which case:

Reading a debuggable app's own files

```
adb shell run-as com.noshorts.blocker ls /data/data/com.noshorts.blocker/shared_prefs
adb shell run-as com.noshorts.blocker cat /data/data/com.noshorts.blocker/shared_prefs/noshorts.xml
```

This works with `NoShorts-debug.apk` and not with the release build, because `run-as` checks the `debuggable` flag baked into the manifest at build time.

adb shell — a terminal on the phone

Android is Linux. `adb shell` with no arguments drops you into an interactive prompt on the device; with arguments it runs one command and returns.

```
adb shell                # interactive; type exit to leave
adb shell ls /sdcard     # one command, then back to your PC
```

GOTCHA — QUOTING IS PARSED TWICE

Your PC shell parses the line, then the device shell parses what survives. Anything with spaces, quotes, `*`, `>` or `&` needs quoting for both. When in doubt, wrap the whole remote command in single quotes: `adb shell 'ls /sdcard/*.xml'`. Without them the `*` expands on your PC, against your PC's files, and the result is nonsense.

The five subsystems worth knowing

PM — PACKAGE MANAGER

```
adb shell pm list packages          # every package
adb shell pm list packages -3       # third-party only
adb shell pm list packages | grep youtube # find one
adb shell pm path com.google.android.youtube # where its APK lives
adb shell pm clear com.noshorts.blocker    # wipe its data, keep it installed
adb shell pm disable-user com.example.bloat # disable without uninstalling
```

AM — ACTIVITY MANAGER

```
adb shell am start -n com.noshorts.blocker/.MainActivity
adb shell am start -a android.intent.action.VIEW -d "https://youtube.com/shorts/abc123"
adb shell am force-stop com.google.android.youtube
```

DUMPSYS — THE STATE OF EVERYTHING

Every system service can dump its internal state. This is how we confirmed the accessibility service was actually bound.

```
adb shell dumpsys accessibility          # our service, bound or not
adb shell dumpsys package com.noshorts.blocker | grep versionName
adb shell dumpsys activity activities | grep topResumedActivity # what is on screen
adb shell dumpsys battery
adb shell dumpsys window displays
```

INPUT – SYNTHETIC TAPS AND KEYS

```
adb shell input tap 540 1200
adb shell input swipe 540 1800 540 600 300      # x1 y1 x2 y2 duration_ms
adb shell input text "hello"                    # no spaces without quoting
adb shell input keyevent KEYCODE_BACK
adb shell input keyevent KEYCODE_HOME
```

WM AND SCREENCAP

```
adb shell wm size                               # Physical size: 1080x2392
adb shell wm density
adb shell screencap -p /sdcard/shot.png && adb pull /sdcard/shot.png
adb shell screenrecord /sdcard/clip.mp4         # Ctrl+C to stop
```

Settings, permissions, appops

This is where adb stops being convenient and starts being genuinely more capable than the UI.

settings

Android keeps configuration in three namespaces: `system` (per-user preferences), `secure` (per-user, app-protected), and `global` (device-wide). The shell user holds `WRITE_SECURE_SETTINGS`, which no ordinary app can obtain.

```
adb shell settings get secure enabled_accessibility_services
adb shell settings put secure accessibility_enabled 1
adb shell settings list global | grep animator
```

That first namespace is exactly how we enabled NoShorts without fighting Android 16's Restricted Settings screen:

Enabling an accessibility service from the command line

```
adb shell settings put secure enabled_accessibility_services \
    com.noshorts.blocker/com.noshorts.blocker.ShortsBlockerService
adb shell settings put secure accessibility_enabled 1
```

GOTCHA — THIS KEY IS A LIST, AND PUT REPLACES IT

`enabled_accessibility_services` holds colon-separated entries. Writing to it *overwrites* the lot, silently disabling any other accessibility service the user depends on. Always read it first. We checked and found it empty before writing, which is the only reason a blind `put` was safe.

Permissions and appops

Runtime permissions, the ones with a dialog, are managed with `pm`:

```
adb shell pm grant com.example android.permission.CAMERA
adb shell pm revoke com.example android.permission.CAMERA
adb shell dumpsys package com.noshorts.blocker | grep -A20 "requested permissions"
```

Special access rights such as "draw over other apps" are not ordinary permissions. They live in a separate subsystem called **app ops**:

```
adb shell appops set com.noshorts.blocker SYSTEM_ALERT_WINDOW allow
adb shell appops get com.noshorts.blocker SYSTEM_ALERT_WINDOW
```

Developer speed-ups

Disabling animations makes a phone feel twice as fast

```
adb shell settings put global window_animation_scale 0
adb shell settings put global transition_animation_scale 0
adb shell settings put global animator_duration_scale 0
# back to normal: set each to 1
```

GOTCHA – CHANGES OUTLIVE THE CABLE

Everything in this section persists after you unplug and after a reboot. adb is not a remote control that stops when you disconnect; it is making the same edits the Settings app would. Write down what you change, or you will spend an evening wondering why animations are gone.

logcat, and when it goes silent

Every app and system service writes to a shared ring buffer. `adb logcat` reads it. Unfiltered it is a firehose, so the flags matter more than the command.

The flags you will actually use

```
adb logcat                # live tail, Ctrl+C to stop
adb logcat -d             # dump what is buffered, then exit
adb logcat -t 50          # last 50 lines only
adb logcat -c             # clear the buffer
adb logcat -s NoShorts    # only the tag NoShorts
adb logcat -d -s NoShorts -t 40 # the combination worth memorising
```

A filter is `TAG:PRIORITY`, where priority is one of Verbose, Debug, Info, Warn, Error, Fatal, or Silent. `-s NoShorts` is shorthand for "silence everything, then show that tag".

```
adb logcat NoShorts:D *:S    # exactly what -s does
adb logcat *:E              # errors from everything
adb logcat -b crash         # the crash buffer
adb logcat --pid=$(adb shell pidof com.noshorts.blocker)
```

In code, the tag is the first argument you pass:

`ShortsBlockerService.kt`

```
Log.i(TAG, "Shorts player detected; pressing Back") // TAG = "NoShorts"
```

GOTCHA — COLOROS SILENTLY DROPS APP LOGS

On your CPH2717, `adb logcat -s NoShorts` returns **nothing at all**, not even the line logged when the service starts. The service is demonstrably running. Several OEM skins, ColorOS and MIUI among them, suppress third-party app logs from the buffer unless logging is enabled in an engineering menu.

This cost us real time during debugging: an empty log looks exactly like code that never ran. If logcat is silent on a device where you know your code is executing, do not conclude the code is dead. Get your evidence another way, which is what Part 09 is for.

Reading the screen: uiautomator

This is the command that actually solved NoShorts, and almost nobody outside test automation knows it exists.

`uiautomator dump` serialises the entire accessibility tree of whatever is currently on screen into XML: every view, its resource id, its class, its bounds, and whether it is on screen. It is the same data an accessibility service sees, which makes it the ground truth for anything built on one.

Capturing what is on screen right now

```
adb shell uiautomator dump /sdcard/ui.xml
adb pull /sdcard/ui.xml
```

With no path it writes to `/sdcard/window_dump.xml`. A node looks like this:

One node from the real Shorts capture

```
<node index="0" class="androidx.recyclerview.widget.RecyclerView"
  package="com.google.android.youtube"
  resource-id="com.google.android.youtube:id/reel_recycler"
  bounds="[0,120][1080,2116]"
  scrollable="true" enabled="true" />
```

`bounds` reads `[left,top][right,bottom]` in pixels. Height here is 2116 minus 120, so 1996 px on a 2392 px screen: 83% of the display, which is what a full-screen player looks like.

The diff that found the bug

Two captures, one with the Shorts player open and one after a single Back press, then compare which `reel_*` ids survive.

STATE	REEL IDS PRESENT
Shorts player open	14, including <code>reel_recycler</code> , <code>reel_watch_player</code> , <code>reel_player_page_container</code>
Home feed, one Back later	1 — <code>reel_time_bar</code> , at bounds <code>[0,0][1080,2392]</code>

That single surviving node was the entire bug. It was in the detection list, it outlives the player, and it reports the full screen, so it passed every "is the player showing" test we threw at it. Detection stayed true on the home feed and Back kept firing until YouTube closed.

METHOD

Three fixes were attempted from memory of YouTube's internals and all three failed. One capture, taken thirty seconds after finally reaching for this command, gave the answer outright. When you are reasoning about another app's UI, measure it. The tools are free and the guessing is not.

Extracting ids and bounds from a dump, PowerShell

```
$xml = [xml](Get-Content ui.xml)
$xml.SelectNodes('//node') |
  Where-Object { $_.'resource-id' -match 'reel' } |
  ForEach-Object { "{0} {1}" -f $_.'resource-id', $_.bounds }
```

Wireless adb

Android 11 and later can do all of this with no cable. The phone and PC must be on the same network.

Android 11+, the modern pairing flow

*# On the phone: Developer options, Wireless debugging, Pair device with pairing code.
It shows an IP:PORT and a six-digit code.*

```
adb pair 192.168.1.42:37105      # then type the code when prompted
adb connect 192.168.1.42:5555  # port from the Wireless debugging main screen
adb devices                    # should list an ip:port entry

adb disconnect 192.168.1.42:5555
```

Note the two different ports. Pairing uses a temporary one that changes each time; connecting uses the persistent one shown on the Wireless debugging screen.

Pre-Android 11, needs one cable to start

```
adb tcpip 5555                  # with USB connected
# unplug now
adb connect 192.168.1.42:5555
adb usb                         # back to cable mode
```

GOTCHA — THIS IS A NETWORK SERVICE

Wireless debugging opens a port on the phone. On a home network that is a reasonable risk; on café or campus Wi-Fi, anyone who can reach the port and get you to accept a prompt has shell access to your device. Turn it off when you are done, and never leave `adb tcpip` mode on a phone you carry around.

Every command this project used

The complete working set from building and debugging NoShorts, in the order it came up. On Windows, prefix each with `F:\platform-tools\adb.exe` until adb is on your PATH.

SETUP AND SANITY

COMMAND	WHY
<code>adb devices -l</code>	Confirm the phone is visible and authorised.
<code>adb kill-server</code>	First move when adb behaves oddly.
<code>adb shell getprop ro.build.version.release</code>	Android version. Told us this was Android 16.
<code>adb shell wm size</code>	Screen size. Needed to turn bounds into percentages.

INSTALL CYCLE

COMMAND	WHY
<code>adb install -r NoShorts-release.apk</code>	Sideload, skipping the Play Protect gate.
<code>adb uninstall com.noshorts.blocker</code>	Required before a differently signed build.
<code>adb shell dumpsys package com.noshorts.blocker grep versionName</code>	Prove the new build actually landed.

ENABLING THE SERVICE

COMMAND	WHY
<code>adb shell settings get secure enabled_accessibility_services</code>	Read before writing. Never skip this.
<code>adb shell settings put secure enabled_accessibility_services <pkg>/<class></code>	Enable, bypassing Restricted Settings.
<code>adb shell settings put secure accessibility_enabled 1</code>	Master switch.
<code>adb shell appops set com.noshorts.blocker SYSTEM_ALERT_WINDOW allow</code>	Draw-over-apps, for the shelf overlays.
<code>adb shell dumpsys accessibility grep -i noshorts</code>	Confirm the framework actually bound it.

FINDING THE BUG

COMMAND	WHY
<code>adb logcat -d -s NoShorts -t 40</code>	Intended diagnostic. Returned nothing on ColorOS.
<code>adb shell dumpsys activity activities grep topResumedActivity</code>	What is actually on screen right now.
<code>adb shell uiautomator dump /sdcard/ui.xml</code>	The command that solved it.
<code>adb pull /sdcard/ui.xml</code>	Bring the tree over to analyse.
<code>adb shell input keyevent KEYCODE_BACK</code>	Change state between captures without touching the phone.

UNDO

To reverse everything this project changed on the handset: `adb shell settings put secure accessibility_enabled 0`, then `adb uninstall com.noshorts.blocker`. To cut the PC off entirely, use Developer options, Revoke USB debugging authorisations.

Troubleshooting, and staying safe

SYMPTOM	MOST LIKELY CAUSE
Device not listed at all	Charge-only cable. Try another before anything else.
unauthorized	RSA prompt not accepted. Replug and watch the screen.
offline	Stale server. <code>adb kill-server</code> , then replug.
INSTALL_FAILED_UPDATE_INCOMPATIBLE	Different signing key. Uninstall first.
INSTALL_FAILED_USER_RESTRICTED	OEM "Install via USB" toggle is off.
INSTALL_FAILED_INSUFFICIENT_STORAGE	Exactly what it says.
more than one device	Emulator running too. Use <code>-s <serial></code> .
Path errors mentioning Program Files	Git Bash rewriting <code>/sdcard</code> . Use PowerShell or <code>//sdcard</code> .
logcat returns nothing	OEM log suppression. Not proof your code did not run.
Accessibility toggle greyed out	Android 13+ Restricted Settings. Allow it in app info, or use <code>settings put</code> .
Service dies after a reboot	Battery optimisation. Set the app to Unrestricted.

Three habits worth keeping

Read before you write. Settings keys are often lists, and `put` replaces rather than appends. One unchecked write to `enabled_accessibility_services` can disable the screen reader someone actually depends on.

Treat USB debugging as a standing key to the device. An authorised computer can install apps, read logs and change secure settings without any further prompt. If you plug into a machine you do not control, revoke it afterwards.

When the phone disagrees with your mental model, dump the phone. `dumpsys` and `uiautomator dump` exist so you never have to argue from memory about what is on screen. Three failed fixes in this project were all the same mistake: reasoning about YouTube's internals from recollection instead of measuring them.